Dhaka International university

Lab Report

Lab Report Topic : Implementation of Page Replacement Algorithms (FIFO, LRU, Optimal)

Lab Report No : 06

Course Title : Operating system lab

Course Code :0613-301

Submitted By:	Submitted To:
Md.Yeasin Bhuiyan Roll:43 Reg No : CS-D-91-23-124699	Md. Nazmus sakib, Lecturer of CSE Department

Date of Submission:27-01-2026

Objectives:

The main objectives of this lab are:

1. **To understand the concept of virtual memory and page replacement.**
2. **To implement FIFO, LRU, and Optimal page replacement algorithms.**
3. **To compare the performance of different page replacement techniques.**
4. **To calculate the number of page faults for each algorithm using C++.**

Background Study (with Example):

In an operating system, page replacement algorithms are used when a page fault occurs and memory is full. The OS must decide which page to remove from memory to load a new page.

1. FIFO (First In First Out):

- **The oldest page in memory is replaced first.**
- **Simple but may cause more page faults.**

Example:

Pages: 7 0 1 2 0 3 0
Frames: 3
Oldest page is replaced whenever memory is full.

2. LRU (Least Recently Used):

- **The page that has not been used for the longest time is replaced.**
- **Better than FIFO but needs tracking of usage.**

Example:

The page least recently accessed is removed when a new page is needed.

3. Optimal Page Replacement:

- **Replaces the page that will not be used for the longest period in the future.**
- **Gives minimum page faults.**

- **Used only for theoretical comparison (future knowledge required).**

Code (with Results):

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// FIFO Algorithm
void FIFO(vector<int> pages, int frames) {
  vector<int> memory;
  int pageFaults = 0;

  for (int page : pages) {
    if (find(memory.begin(), memory.end(), page) == memory.end()) {
      if (memory.size() == frames)
        memory.erase(memory.begin());
      memory.push_back(page);
      pageFaults++;
    }
  }
  cout << "FIFO Page Faults: " << pageFaults << endl;
}

// LRU Algorithm
void LRU(vector<int> pages, int frames) {
  vector<int> memory;
  int pageFaults = 0;
```

```
    for (int i = 0; i < pages.size(); i++) {
        auto it = find(memory.begin(), memory.end(), pages[i]);
        if (it == memory.end()) {
            if (memory.size() == frames)
                memory.erase(memory.begin());
            memory.push_back(pages[i]);
            pageFaults++;
        } else {
            memory.erase(it);
            memory.push_back(pages[i]);
        }
    }
    cout << "LRU Page Faults: " << pageFaults << endl;
}

// Optimal Algorithm
void Optimal(vector<int> pages, int frames) {
    vector<int> memory;
    int pageFaults = 0;

    for (int i = 0; i < pages.size(); i++) {
        if (find(memory.begin(), memory.end(), pages[i]) == memory.end()) {
            if (memory.size() < frames) {
                memory.push_back(pages[i]);
            } else {
                int idx = -1, farthest = i + 1;
                for (int j = 0; j < memory.size(); j++) {
```

```
            int k;
            for (k = i + 1; k < pages.size(); k++) {
              if (memory[j] == pages[k]) break;
            }
            if (k > farthest) {
              farthest = k;
              idx = j;
            }
          }
          if (idx == -1)
            idx = 0;
          memory[idx] = pages[i];
        }
        pageFaults++;
      }
    }
    cout << "Optimal Page Faults: " << pageFaults << endl;
}

int main() {
    int n, frames, choice;
    cout << "Enter number of pages: ";
    cin >> n;

    vector<int> pages(n);
    cout << "Enter page reference string:\n";
    for (int i = 0; i < n; i++)
      cin >> pages[i];
```

```
    cout << "Enter number of frames: ";
    cin >> frames;

    while (true) {
        cout << "\nMenu\n";
        cout << "1. FIFO\n2. LRU\n3. Optimal\n4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: FIFO(pages, frames); break;
            case 2: LRU(pages, frames); break;
            case 3: Optimal(pages, frames); break;
            case 4: exit(0);
            default: cout << "Invalid Choice\n";
        }
    }
    return 0;
}
```

Output:

FIFO Page Faults: 7

LRU Page Faults: 6

Optimal Page Faults: 5

{P

Discussion:

In this lab, three page replacement algorithms were implemented and compared. FIFO is simple but inefficient in many cases. LRU performs better by considering

recent usage. Optimal gives the best result with minimum page faults but cannot be implemented practically because it requires future knowledge. Among all, Optimal performs best, followed by LRU, while FIFO gives the highest number of page faults.